

Pivot-Constrained Trendline Trading (PCTT): A Research-Grade, Production-Ready Framework

Author: Experienced Trader and Financial Mathematical Engineer

Date: 2026-01-17

Version: v2.0 (Production + Live Deployment)

Keywords: trendlines, pivots, market structure, regime detection, event state machine, risk management, execution, calibration, statistical validation, Pine Script, TradingView automation

Abstract

Pivot-Constrained Trendline Trading (PCTT) is a systematic market-structure methodology that converts discretionary trendline analysis into a deterministic, testable, automatable trading system. PCTT defines structural boundaries using confirmed pivots, scores their reliability (Q-score), classifies market regimes to avoid chop, and detects tradeable events (break, freeze, retest, failure) via a non-repainting state machine. This paper presents the fundamental theory, a quantitative specification, and a production deployment design spanning TradingView chart overlays, webhook ingestion, broker-neutral execution, and institutional-grade risk governance. We further incorporate corrections and enhancements: robust boundary estimation with regularization and outlier resistance, mandatory probability calibration for Q-scores, microstructure-aware sizing and execution realism, and a statistical validation suite (bootstrap, permutation tests, Monte Carlo, and White-type reality checks). Finally, we outline a TradeZella-plus analytics stack for post-trade attribution, setup taxonomy, and drift monitoring, plus an instrument selection and rotation engine that selects markets and symbols most compatible with PCTT structure at any moment.

1. Introduction

Trendlines are among the most widely used tools in technical analysis because they compress market structure into actionable geometry: support, resistance, channels, and wedges. Yet traditional trendline trading is plagued by subjectivity (different traders draw different lines), non-reproducibility (signals cannot be recreated reliably), and lookahead bias (historical lines change when re-drawn). PCTT addresses these issues by replacing hand-drawn lines with pivot-constrained boundaries selected by objective scoring, and by treating structural events as state transitions rather than informal chart annotations.

The goal is not to claim that PCTT guarantees any win rate across all markets. No legitimate framework can guarantee 85-90 percent wins without restricting conditions and accepting trade-offs. The goal is to create an engineering-grade system that: (i) reduces ambiguity, (ii) filters low-quality regimes, (iii) contains downside via robust risk management, (iv) produces stable signals suitable for backtesting and automation, and (v) continuously validates itself via calibration and statistical tests.

2. Conceptual Overview: From Discretionary Lines to Structure Objects

PCTT treats market structure as a first-class object. At any bar t we maintain a Structure Object S_t that includes the current support and resistance boundaries, quality scores, regime label, and event state. Downstream components (entry logic, risk, execution, analytics) consume S_t in a broker- and market-agnostic way.

2.1 Structure Object Definition

Let price series be P_t with OHLCV and optional microstructure fields. Define the PCTT Structure Object as:

$$S_t = (L_t, U_t, Q_{L,t}, Q_{U,t}, R_t, E_t, d_{L,t}, d_{U,t}, \text{meta}_t)$$

where L_t and U_t are lower and upper boundaries (support/resistance), Q scores in $[0,1]$ quantify reliability, R_t is regime (Trend/Range/Transition), E_t is the event state (Idle/Break/Freeze/Retest/Failure/Trade), d_* are ATR-normalized distances from current price to boundaries, and meta_t holds auxiliary features (ATR, spread proxy, session flags, etc.).

2.2 Action Line and Safety Line

PCTT distinguishes two roles: the Action Line (the boundary being interacted with for an entry trigger) and the Safety Line (the opposite boundary used for risk geometry, invalidation, and trailing). In an uptrend channel, a long entry might be triggered by a resistance break (action = U_t), while safety = L_t provides a structural stop reference. After a confirmed break, both lines are frozen to prevent moving-goalpost bias.

3. Data Representation and Normalization

A production framework must be scale-invariant: a 2 dollar move on a 10 dollar stock differs from a 2 dollar move on BTC. PCTT uses ATR-normalized geometry in price space (practical for Pine Script), with optional log-price modeling for research backtests.

3.1 ATR-Normalized Distances

Define $\text{ATR}_t = \text{ATR}(P, n)$. Any raw distance d in price units becomes a dimensionless distance: $d_{\text{tilde}} = d / \text{ATR}_t$. All buffers (penetration, close confirmation, touch tolerance) are defined in ATR units to self-scale across volatility regimes.

3.2 Microstructure Proxies

If true bid-ask data is unavailable (common on TradingView), we approximate friction using proxies: (i) high-low range vs close-to-close volatility, (ii) volume-weighted spread estimates for stocks, (iii) funding rates for crypto perps, and (iv) broker-reported spread where available. These proxies enter the cost penalty in scoring, and the risk engine inflates stop buffers when friction is high.

4. Pivot Extraction: Deterministic Structural Points

Pivots anchor structure. A pivot low at time i is confirmed after k bars on both sides close above it (and analogously for pivot highs). Confirmation lag is a feature, not a bug: it eliminates lookahead bias and ensures non-repainting pivot sets.

4.1 Pivot Definition

Let k be pivot depth. A pivot low occurs at i if Low_i is the minimum in $[i-k, i+k]$. In Pine, `ta.pivotlow` confirms only after k bars of future data, which is safe because the pivot timestamp is fixed once confirmed.

4.2 Degenerate Cases

Production code must handle degenerate cases: insufficient pivots, clustered pivots, and monotonic markets with no swings. PCTT imposes a minimum pivot count (e.g., $n \geq 5$ in lookback) before a structure is considered tradable. If n is lower, the system either widens timeframe or switches instruments (via the instrument engine).

5. Boundary Estimation: Pivot-Constrained Candidate Search

Boundaries are estimated by generating candidate lines from pivot pairs and selecting the best line by an objective function. This eliminates manual drawing and provides reproducible results.

5.1 Candidate Line Parameterization

For any two pivots (t_1, p_1) and (t_2, p_2) , a line is defined by slope $m = (p_2 - p_1) / (t_2 - t_1)$ and intercept $b = p_1 - m t_1$ in bar-index space. The projected line value at time t is $l(t) = m t + b$.

5.2 Robust Objective Function

A naive least-squares fit is fragile because price contains outliers (wicks, news spikes). PCTT uses robust scoring that rewards touches and penalizes violations with ATR-normalized magnitudes.

Let T be the set of confirmed pivots relevant to the candidate. Define a touch if a pivot lies within tolerance eps_touchATR of the line. Define a violation if close crosses beyond the line by eps_breakATR in the wrong direction.

We define a raw score as:

$$\text{Score} = w_{\text{touch}} \sum_j \phi_{\text{touch}}(d_j) - w_{\text{viol}} \sum_j \phi_{\text{viol}}(v_j) + w_{\text{persist}} \log(1 + \Delta T) - w_{\text{slope}} |m|$$

where d_j is pivot-to-line distance in ATR units, v_j is violation depth in ATR units, ΔT is time span between defining pivots, and w_{slope} is a ridge-style regularization discouraging extreme slopes in noisy

markets.

5.3 Outlier Resistance: Huber and RANSAC Enhancements

Two complementary robustness layers are recommended. First, replace squared penalties with Huber-type losses so large deviations do not dominate. Second, require a minimum inlier consensus: the line must be within tolerance for at least c pivots (e.g., 3) to be valid. A RANSAC-like routine chooses the best candidate among those with sufficient consensus, reducing sensitivity to single anomalous pivot prints.

5.4 Convergence and Stability

Although the candidate search is finite (bounded by pivot pairs), stability still matters: small data changes should not cause frequent boundary jumps. We enforce stability with: (i) hysteresis on boundary changes (only accept a new best line if its score exceeds the current by Δ_{score}), and (ii) a minimum life requirement (line must persist M bars before being tradable unless score is exceptional).

6. Line Quality Scoring: Q-Score and Mandatory Calibration

The Q-score compresses structural reliability into $[0,1]$. But in production, Q must be calibrated to real probabilities: a Q of 0.80 should correspond to an empirically observed success probability near 0.80 under similar conditions.

6.1 From Raw Score to Q

We map raw Score to Q via a sigmoid:

$$Q = 1 / (1 + \exp(-\text{Score}))$$

This gives a smooth bounded confidence measure, but it is not automatically calibrated.

6.2 Calibration (Required, not Optional)

Production requires ongoing calibration. Use isotonic regression or Platt scaling to map Q to calibrated probability p_{hat} . Recalibrate on a rolling basis (e.g., every 500 bars or monthly), with a minimum sample size (e.g., ≥ 200 trades). Monitor Brier score and reliability diagrams; if calibration degrades beyond thresholds, reduce size or disable auto-rotation until retrained.

7. Regime Detection: Avoiding Chop and Structural Noise

Trendline strategies often die in range-bound chop. PCTT gates signals using regime classification based on directional efficiency and crossing behavior.

7.1 Efficiency Ratio (ER)

ER over window N is $|P_t - P_{t-N}| / \sum_{i=1..N} |P_{t-i+1} - P_{t-i}|$. ER near 1 indicates efficient directional travel; near 0 indicates choppy mean reversion.

7.2 Crossing Count and Midline Choice

Define a midline M_t (e.g., EMA or channel midpoint). Count the number of times price crosses M_t within N bars. High crossing count implies range. Combined with ER, we define regimes: Trend = ER high AND crossings low; Range = ER low OR crossings high; Transition otherwise.

7.3 Optional: Hidden Markov Model Layer

For advanced deployments, wrap ER and crossings into an HMM with three states (Trend/Range/Transition) to reduce regime flip-flopping. The HMM provides smoothed regime probabilities and can be trained per instrument class.

8. Event Detection via State Machine (Non-Repainting)

The PCTT alpha is not merely a line; it is the sequence of events: Break -> Freeze -> Retest -> Acceptance/Failure. A formal state machine ensures signals are time-consistent and backtest-safe.

8.1 Two-Stage Break Confirmation

A break event requires (i) penetration by wick beyond the line by β_{pATR} and (ii) close confirmation beyond the line by β_{cATR} . This rejects many false breaks caused by wicks.

8.2 Freeze (Action and Safety Line Lock)

At the first confirmed break bar t_0 , store the action line parameters (m,b) and the safety line parameters. For subsequent bars, project them linearly. Retests must occur against the frozen line, not a recomputed moving boundary.

8.3 Retest Window and Rejection Score

After freeze, open a retest window of M bars. A retest occurs if price returns to within $\epsilon_{retest} * ATR$ of the action line. Retest acceptance is quantified by a Rejection Score that measures whether candles reject the line in the intended direction (e.g., lower wicks + closes away from line for longs).

8.4 Failure and Reset Logic

If price invalidates the structure by closing beyond the safety line (or violates the broken line by $\gamma_{fail} * ATR$), the state transitions to Failure and then resets to Idle after cooldown. This prevents re-entry spam and enforces one-break-one-trade discipline.

9. Trade Construction: Entries, Exits, and Risk Geometry

A PCTT trade is defined by the structural geometry between action and safety lines plus ATR buffers. The entry is typically after retest confirmation, not at the initial break.

9.1 Entry Rules (Canonical)

Long setup example: (1) regime is Trend or Transition, (2) resistance breaks with two-stage confirmation, (3) freeze action/safety lines, (4) retest occurs within M bars, (5) rejection score above threshold, (6) Q-score above threshold, (7) costs acceptable. Short setups invert conditions.

9.2 Stop Placement (Structural First)

Primary stop uses the safety line with buffer: $\text{stop} = \text{safety_line} - k_{\text{stop}} \times \text{ATR}$ for longs (plus spread/slippage buffer). If safety line is too far (poor risk geometry), the trade is rejected or position sized down.

9.3 Targets and Partial Exits

Define $1R = \text{entry} - \text{stop}$ (in price). Use partial exits at $1R$ (e.g., 50 percent) to reduce tail risk, then trail the remainder structurally. Targets can be static multiples (2R, 3R) or structure-based (next boundary, measured move, volatility bands).

10. Robust Risk Management (Production-Grade)

Risk management is the difference between a clever idea and a survivable business. PCTT uses layered defense: per-trade risk, portfolio heat, drawdown throttles, microstructure-aware buffers, and circuit breakers.

10.1 Per-Trade Risk and Position Sizing

Size is determined by risk budget and stop distance: $\text{qty} = (\text{account_equity} \times r_{\text{pct}}) / (\text{stop_distance} + \text{friction_buffer})$. For instruments with varying tick sizes (futures) or contract multipliers, convert stop_distance to currency risk correctly.

10.2 Portfolio Heat and Correlation Controls

Define heat as sum of worst-case losses of open positions at their stops divided by equity. Enforce max heat (e.g., 6 percent). Correlation control blocks new entries if they increase exposure to highly correlated symbols (e.g., multiple tech stocks, crypto beta basket). Use rolling return correlation or factor models.

10.3 Drawdown-Based Scaling and Kill Switches

Scale down risk when drawdown exceeds thresholds (e.g., halve risk at 5 percent DD, stop trading at 10 percent DD until recovery). Include a global kill switch for broker outages, abnormal slippage, or drift alarms.

10.4 Trailing Stop System (Hybrid Structural Trail)

A robust trailing system should (i) protect capital, (ii) avoid being shaken out by noise, and (iii) adapt to structure. PCTT uses a hybrid trail:

Stage A - Structural stop: initial stop anchored to safety line.

Stage B - Break-even lock: once trade reaches +bR (e.g., 0.8R), move stop to entry plus friction.

Stage C - Partial exit: take partial at 1R and reduce risk.

Stage D - Pivot trail: trail stop behind higher lows (longs) or lower highs (shorts) using confirmed pivots and ATR buffer.

Stage E - Time stop: exit if trade fails to progress within T bars.

10.5 Avoiding False Confidence

Trailing stops cannot guarantee a minimum win rate. They shift the distribution: fewer large losses, potentially more small losses, often higher win rate but sometimes lower expectancy. Production tuning should maximize expectancy and drawdown-adjusted returns, not only win rate.

11. Microstructure and Execution Realism

Ignoring execution turns backtests into fiction. PCTT production design accounts for spread, slippage, market impact, partial fills, and adverse selection.

11.1 Spread and Slippage Modeling

Use dynamic slippage: $\text{slippage} = a + b (\text{volatility proxy}) + c (\text{order size} / \text{ADV})$. For crypto, include fee tiers and funding. For futures, include tick size and contract multiplier.

11.2 Order Type Logic

Define execution style per setup: market orders on momentum breaks; limit orders on retest entries; stop-limit cautiously when slippage can explode. Include order timeouts and fallback to market if not filled within acceptable window.

11.3 Partial Fills and Reconciliation

OMS tracks order state (submitted/partial/filled/cancelled/rejected). Reconcile internal positions to broker truth on a schedule and after disconnects.

12. Indicators That Enhance PCTT Without Diluting It

PCTT is structure-first. Indicators are used as context gates, not as primary triggers. The best enhancements are those that detect trend health, volatility regime, and exhaustion.

12.1 Trend Health Indicators

Recommended: ER, ADX, moving-average slope normalized by ATR, and price position relative to anchored VWAP (stocks) or session VWAP (futures).

12.2 Reversal / Turnaround Context

Use pivot-based swing structure, momentum divergence (RSI/MACD), volatility contraction then expansion, and failed-break patterns (break then immediate reclaim). In PCTT, a failure state itself is a reversal signal candidate, but should be gated by regime and costs.

12.3 Volume and Order Flow Proxies

For stocks/futures: volume profile zones, VWAP deviations, and relative volume (RVOL). For crypto: volume spikes, open interest changes, and funding extremes. These features improve instrument selection and risk buffers.

13. Instrument Selection and Switching Engine (Tiered)

PCTT performance depends on trading instruments that exhibit clean structure. The instrument engine ranks symbols by (i) eligibility constraints, (ii) momentum strength, (iii) PCTT readiness (Q-score and event proximity), (iv) cost penalty, and (v) learned edge per symbol.

Auto-rotation uses hysteresis and cooldowns to prevent churn. A symbol is promoted only if its score remains in the top K for M bars and exceeds the weakest active symbol by a score-gap threshold. A demoted symbol is placed on cooldown.

14. Statistical Validation Suite (Required for Research-Grade Claims)

A production strategy must be validated against data-mining bias and regime overfit. This section outlines tests that should be automated within the analytics stack.

14.1 Bootstrap Confidence Intervals

Compute block bootstrap confidence intervals for expectancy, Sharpe, win rate, profit factor, and max drawdown. Use blocks to preserve serial dependence.

14.2 Permutation and Significance Testing

Use permutation tests on entry timestamps (or signal labels) to estimate how often similar performance appears by chance. For multiple strategies/parameter sets, apply White-type reality checks to control for data snooping.

14.3 Monte Carlo Trade Sequencing

Shuffle trade outcomes (or sample from fitted distributions) to simulate equity curves under variability. Compute risk of ruin, drawdown distributions, and required capital buffers.

14.4 Walk-Forward and Drift Monitoring

Run walk-forward tests by recalibrating Q and re-optimizing thresholds on rolling windows, then evaluating on forward windows. In live trading, monitor drift: calibration Brier score, hit rate by regime, and slippage anomalies.

15. Technical Implementation: TradingView + Pine Script + Production Engine

PCTT can be deployed as: (A) chart-native visual overlays and alerts in TradingView, (B) a backend execution engine that consumes webhook signals, applies risk governance, and routes orders to brokers, and (C) a UI trading terminal for monitoring and analytics.

15.1 TradingView Visual Layer (The Wow Effect)

The chart should display: pivots, candidate structure, selected action/safety lines, frozen lines after break, retest window shading, and event badges. Agents can be visualized as small status widgets: Regime Agent, Structure Agent, Risk Agent, Execution Agent.

15.2 Webhook Payload Contract

Alerts must include enough context for deterministic execution: symbol, timeframe, bar time, event type, direction, entry reference, action/safety lines, Q, regime metrics, and risk geometry. Use HMAC signing and idempotency keys.

15.3 Backend Engine: State, Risk, Execution, Reconciliation

The backend must be server-authoritative. TradingView alerts are inputs; the backend validates regime, cost, risk, portfolio constraints, and broker availability before placing orders. Positions are reconciled against broker truth to survive disconnects and partial fills.

16. Practical Application by Market

PCTT is market-agnostic in theory but must adapt to contract specifications, sessions, and microstructure realities. This section provides practical constraints and tuning defaults.

16.1 Stocks

Stocks have session gaps, earnings risk, and liquidity dispersion. Use strict eligibility: dollar volume, spread, and earnings blackout filters. Favor 15m-4h timeframes for structure, with regime gating to avoid midday chop. Use anchored VWAP and RVOL as context gates.

Execution should prefer limit orders on retests. Position sizing should incorporate ADV and potential gap risk: reduce risk or require options hedges when holding overnight. Corporate actions must be handled (splits, halts).

16.2 Futures

Futures require contract multipliers, tick sizes, and roll logic. Prefer front-month liquid contracts and define session windows (e.g., RTH vs ETH). Use volume and VWAP context heavily.

Stops must respect tick size and exchange rules. Slippage can spike at opens and news releases; add time-of-day volatility buffers. Margin and overnight maintenance requirements should be enforced by the risk governor.

16.3 Options

Options selection is a second-layer problem. Use PCTT to pick underlyings and direction; then construct option positions (vertical spreads, calendars) according to IV regime, delta targets, and DTE bands.

Risk should be defined in premium terms and Greeks; trailing stops are often implemented as delta-based or underlying-structure-based exits. Liquidity checks (OI, bid-ask) are mandatory.

16.4 Forex

Forex is 24/5 with spread variability and session-driven behavior. Use session filters (London/NY overlap) and strict spread thresholds. Volume is not reliable on spot feeds, so rely on structure plus volatility proxies.

Pip conversion and leverage controls are critical. News filters for high-impact events reduce tail losses. Consider using ATR buffers scaled up on low-liquidity hours.

16.5 Crypto

Crypto is 24/7 with frequent regime shifts, funding costs, and weekend effects. Eligibility filters include volume, spread proxy, and funding bounds. Prefer higher timeframes (1h-4h) for cleaner pivots.

Use exchange fees and potential liquidation cascades as risk considerations. Consider volatility shock filters and max leverage caps. Auto-rotation can be effective due to constant relative momentum shifts.

17. TradeZella-Plus Analytics: Instrumented, Explainable, Self-Monitoring

To beat basic journaling tools, analytics must be event-native and structure-aware. Every trade should carry tags derived from PCTT state: regime, Q bucket, break type, retest depth, rejection score, cost grade, instrument tier score, and execution style.

Core analytics epics: (i) expectancy decomposition by regime and setup, (ii) MFE/MAE distributions per setup, (iii) time-in-trade and time-to-1R, (iv) slippage and spread attribution, (v) edge drift monitoring, (vi) walk-forward reports, (vii) portfolio heat and correlation exposure time series, and (viii) instrument rotation audit trails.

18. Failure Modes and Safeguards

Common failure modes include: overtrading during ranges, overfitting thresholds, underestimating costs, ignoring gaps, and assuming Q is calibrated when it is not. Safeguards: regime gating, minimum pivot counts, calibration monitoring, slippage alarms, drawdown scaling, and a kill switch.

19. Reference Pine Script (Research Template)

The following Pine v5 template illustrates the core mechanics: pivot extraction, line projection, break confirmation, freeze, retest window, rejection scoring, and alert payload. It is intentionally modular. Production implementations should optimize loops and cap candidate counts to respect Pine limits.

```
//@version=5
indicator("PCTT Core Template (Research)", overlay=true, max_lines_count=500, max_labels_count=500)

// -----
// Inputs
// -----
leftRight = input.int(5, "Pivot left/right", minval=2, maxval=20)
lookbackPiv = input.int(150, "Pivot lookback bars", minval=50, maxval=2000)
atrLen = input.int(14, "ATR length", minval=5, maxval=100)

// Buffers in ATR units
betaPen = input.float(0.20, "Break penetration (ATR)", step=0.05)
betaCls = input.float(0.30, "Break close confirm (ATR)", step=0.05)
epsTouch = input.float(0.15, "Touch tolerance (ATR)", step=0.05)
epsRetest = input.float(0.20, "Retest tolerance (ATR)", step=0.05)
failBuf = input.float(0.35, "Failure buffer (ATR)", step=0.05)

retestBars = input.int(20, "Retest window bars", minval=3, maxval=100)
minQ = input.float(0.65, "Min Q-score", step=0.01)

// -----
// Helpers
// -----
atr = ta.atr(atrLen)
barIndex = bar_index

// Confirmed pivots (non-repainting, but confirmed with lag)
pl = ta.pivotlow(low, leftRight, leftRight)
ph = ta.pivohigh(high, leftRight, leftRight)

// Store pivots in arrays
var pl_t = array.new_int()
var pl_p = array.new_float()
var ph_t = array.new_int()
var ph_p = array.new_float()

if not na(pl)
    t = barIndex - leftRight
```

```

    array.push(pl_t, t)
    array.push(pl_p, pl)

if not na(ph)
    t = barIndex - leftRight
    array.push(ph_t, t)
    array.push(ph_p, ph)

// Keep arrays bounded
f_trim(arrT, arrP, maxBars) =>
    while array.size(arrT) > 0 and (barIndex - array.get(arrT, 0)) > maxBars
        array.shift(arrT)
        array.shift(arrP)

f_trim(pl_t, pl_p, lookbackPiv)
f_trim(ph_t, ph_p, lookbackPiv)

// Line struct helper
f_line_y(m, b, t) => m * t + b
f_fit_line(t1, p1, t2, p2) =>
    m = (p2 - p1) / math.max(1.0, (t2 - t1))
    b = p1 - m * t1
    [m, b]

// Score candidate line by touches/violations (simplified)
f_score_support(m, b) =>
    touches = 0.0
    viol = 0.0
    // touch pivots
    for i = 0 to array.size(pl_t) - 1
        tt = array.get(pl_t, i)
        pp = array.get(pl_p, i)
        yy = f_line_y(m, b, tt)
        d = math.abs(pp - yy) / atr
        if d <= epsTouch
            touches += (1.0 - d/epsTouch)
    // violations by closes below line
    yyNow = f_line_y(m, b, barIndex)
    v = math.max(0.0, (yyNow - close) / atr)
    if v > 0
        viol += v
    // ridge penalty on slope magnitude
    slopePen = 0.25 * math.abs(m)
    // persistence reward (use span of defining pivots)
    touches - 1.5*viol - slopePen

f_score_resist(m, b) =>
    touches = 0.0
    viol = 0.0
    for i = 0 to array.size(ph_t) - 1
        tt = array.get(ph_t, i)
        pp = array.get(ph_p, i)
        yy = f_line_y(m, b, tt)
        d = math.abs(pp - yy) / atr
        if d <= epsTouch
            touches += (1.0 - d/epsTouch)
    yyNow = f_line_y(m, b, barIndex)
    v = math.max(0.0, (close - yyNow) / atr)
    if v > 0
        viol += v
    slopePen = 0.25 * math.abs(m)
    touches - 1.5*viol - slopePen

```

```

f_sigmoid(x) => 1.0 / (1.0 + math.exp(-x))

// -----
// Select best support/resistance from pivot pairs (cap pairs)
// -----
maxPairs = 15

var float mL = na
var float bL = na
var float qL = na

var float mU = na
var float bU = na
var float qU = na

// Best support
if array.size(pl_t) >= 2
    bestScore = -1e9
    bestM = na
    bestB = na
    // take last maxPairs pivots
    start = math.max(0, array.size(pl_t) - maxPairs)
    for i = start to array.size(pl_t) - 2
        for j = i+1 to array.size(pl_t) - 1
            t1 = array.get(pl_t, i)
            p1 = array.get(pl_p, i)
            t2 = array.get(pl_t, j)
            p2 = array.get(pl_p, j)
            [m, b] = f_fit_line(t1, p1, t2, p2)
            s = f_score_support(m, b)
            if s > bestScore
                bestScore := s
                bestM := m
                bestB := b
    mL := bestM
    bL := bestB
    qL := f_sigmoid(bestScore)

// Best resistance
if array.size(ph_t) >= 2
    bestScore = -1e9
    bestM = na
    bestB = na
    start = math.max(0, array.size(ph_t) - maxPairs)
    for i = start to array.size(ph_t) - 2
        for j = i+1 to array.size(ph_t) - 1
            t1 = array.get(ph_t, i)
            p1 = array.get(ph_p, i)
            t2 = array.get(ph_t, j)
            p2 = array.get(ph_p, j)
            [m, b] = f_fit_line(t1, p1, t2, p2)
            s = f_score_resist(m, b)
            if s > bestScore
                bestScore := s
                bestM := m
                bestB := b
    mU := bestM
    bU := bestB
    qU := f_sigmoid(bestScore)

// Current projected lines
lNow = na(mL) ? na : f_line_y(mL, bL, barIndex)
uNow = na(mU) ? na : f_line_y(mU, bU, barIndex)

```

```

plot(lNow, "Support", color=color.new(color.green, 0), linewidth=2)
plot(uNow, "Resistance", color=color.new(color.red, 0), linewidth=2)

// -----
// State machine: Idle -> Break -> Freeze -> Retest -> Trade/Fail
// -----
var int state = 0
// 0 Idle, 1 BreakUp, 2 FreezeUp, 3 RetestUp, 4 BreakDn, 5 FreezeDn, 6 RetestDn
var int t0 = na
var float fmA = na
var float fbA = na
var float fmS = na
var float fbS = na
var float qA = na

f_freeze(actionM, actionB, safetyM, safetyB, q) =>
    t0 := barIndex
    fmA := actionM
    fbA := actionB
    fmS := safetyM
    fbS := safetyB
    qA := q

f_proj(m, b) => f_line_y(m, b, barIndex)

// Break up of resistance
if state == 0 and not na(uNow) and qU >= minQ
    penOK = (high - uNow) / atr >= betaPen
    clsOK = (close - uNow) / atr >= betaCls
    if penOK and clsOK
        // freeze action=resistance, safety=support
        f_freeze(mU, bU, mL, bL, qU)
        state := 2

// FreezeUp -> wait retest
if state == 2
    bars = barIndex - t0
    aLine = f_proj(fmA, fbA)
    sLine = f_proj(fmS, fbS)
    // retest if price comes back to action line within tolerance
    retest = math.abs(low - aLine) / atr <= epsRetest or math.abs(close - aLine) / atr <= epsRetest
    // failure if close back below safety or deep under action
    fail = (close - sLine) / atr <= -failBuf
    if fail
        state := 0
    else if retest
        state := 3
    else if bars > retestBars
        state := 0

// RetestUp -> simple rejection score (toy)
if state == 3
    aLine = f_proj(fmA, fbA)
    // rejection: close above line and wick below line
    rej = close > aLine and low < aLine
    if rej
        // fire alert
        payload = '{"strategy":"PCTT","event":"ENTRY_LONG","symbol":"' + syminfo.tickerid + '","tf":"' + t
        alert(payload, alert.freq_once_per_bar_close)
        label.new(barIndex, high, "PCTT LONG", style=label.style_label_up)
        state := 0

// NOTE: Symmetric short-side states omitted for brevity in this template.

```

19.1 Pine Script Notes and Production Hardening

This template is intentionally simplified. Production hardening includes: (i) more robust scoring with explicit violation counting over a window, (ii) candidate line stability hysteresis, (iii) full short-side symmetry, (iv) explicit regime gating (ER + crossings), (v) richer rejection scoring (candle anatomy, volume context), (vi) alert idempotency keys, and (vii) strict caps to avoid Pine performance limits.

20. Platform Implementation Notes (Live Trading)

A full production deployment uses TradingView for visualization and alerting, while the backend platform enforces risk and execution realism. Key requirements: HMAC signatures, nonce replay protection, low-latency webhook ingestion (<200 ms), message queue for async processing, broker abstraction layer, OMS state machine, reconciliation workers, and observability (metrics, traces, alerts).

21. Conclusion

PCTT converts a subjective technique into a deterministic system: pivots define candidate boundaries; objective scoring selects robust action/safety lines; Q-score quantifies reliability; calibration ties Q to empirical probabilities; regime detection avoids chop; and a state machine defines break/retest/failure without repainting. In production, the strategy is inseparable from risk governance, execution realism, statistical validation, and analytics. The result is not a promise of guaranteed wins, but a disciplined structure-driven engine that can be deployed across markets, monitored for drift, and integrated into a multi-agent trading platform with live chart-native wow-factor.

References (Selected)

Kaufman, P. (1995). Smarter Trading.

Murphy, J. J. (1999). Technical Analysis of the Financial Markets.

Hamilton, J. (1994). Time Series Analysis (regime models).

Efron, B. and Tibshirani, R. (1993). An Introduction to the Bootstrap.

White, H. (2000). A Reality Check for Data Snooping (econometrics literature).

TradingView Documentation: Pine Script and Charting Libraries.

Appendix A: Production Checklists and Extended Derivations

A.1 Extended Boundary Scoring

A production scorer should evaluate candidate lines over a fixed evaluation window W and compute:

- Touch density: touches per 100 bars
- Touch spacing: entropy or minimum spacing between touches to reduce clustered noise
- Violation depth distribution: mean and 95th percentile in ATR units

- Wick vs close violations: treat close violations as stronger evidence than wick violations

A robust formulation uses Huber loss $L_{\Delta}(x)$ and a ridge penalty on slope magnitude.

Score = $w_1 \text{TouchReward} - w_2 \text{CloseViolation} - w_3 \text{WickViolation} - w_4 \text{HuberResidual} - w_5 |m| + w_6 \log(1 + \Delta T)$.

A.2 Rejection Score (Candle Anatomy)

Define for longs at retest:

- body = $|\text{close} - \text{open}|$
- upperWick = $\text{high} - \max(\text{open}, \text{close})$
- lowerWick = $\min(\text{open}, \text{close}) - \text{low}$
- distanceToLine = $|\text{close} - \text{actionLine}|$

A simple rejection score might be:

$\text{Rej} = \sigma(\text{lowerWick}/\text{body}) \cdot \sigma((\text{close} - \text{actionLine})/\text{ATR}) \cdot \sigma(-\text{distanceToLine}/\text{ATR})$.

For shorts, invert wick and sign. Combine with volume confirmation where available.

A.3 Regime Threshold Selection

Instead of fixed thresholds, estimate ER quantiles per instrument and choose adaptive cutoffs.

Example: Trend if ER > Q70 and crossings < Q30.

Use an HMM or exponentially weighted smoothing to reduce chattering.

A.4 Walk-Forward Protocol

Split history into rolling windows:

- Train window: calibrate Q and optimize thresholds (minQ, betaPen, betaCIs, epsRetest)
- Test window: fixed parameters, no re-optimization

Repeat. Report distribution of results and parameter stability.

A.5 Risk-of-Ruin and Tail Risk

Estimate tail loss probability using empirical MAE and slippage distributions.

Simulate equity paths with Monte Carlo to compute $P(\text{DD} > x)$ and time-to-recovery statistics.

Apply dynamic risk scaling: $\text{risk_pct} = \text{base_risk } f(\text{drawdown}) f(\text{calibration_quality})$.

A.6 Execution Guardrails

Minimum production guardrails:

- Reject entries when spread/ATR exceeds threshold
- Reject during scheduled high-impact events (where supported)
- Stop order validation: ensure stop price respects tick size and broker rules
- Slippage alarms: if realized slippage exceeds expected by N std dev, pause trading
- Reconciliation: compare positions every minute and after reconnects

A.7 Analytics That Beat Basic Journals

TradeZella-plus analytics should support:

- Setup taxonomy: break type, retest depth, rejection score bucket, Q bucket
- Regime attribution: performance by regime and by regime transitions
- Execution attribution: PnL explained by signal quality vs slippage vs spread
- MFE/MAE heatmaps: outcome probability conditioned on early excursion
- Instrument rotation accountability: every rotation with before/after scores
- Drift detection: Brier score, hit rate decay, cost inflation, regime mix changes

A.8 Instrument Selection Engine Notes

Use tiered selection:

- Tier 1 - eligibility filters (liquidity, spread, session)
- Tier 2 - opportunity scoring (momentum + PCTT readiness)
- Tier 3 - symbol-specific edge learning (Bayesian)
- Tier 4 - rotation with hysteresis and cooldown

This engine is not only about finding winners; it is about avoiding instruments where structure is noisy or costs dominate.

A.9.1 Operational Note 1

In live production, the difference between a good system and a brittle one is rarely the entry rule. It is consistency under stress: data outages, latency spikes, broker quirks, session transitions, and user misconfiguration. Implement defensive checks at every boundary: validate inputs, cap loops, sanitize payloads, and fail closed when uncertain. Store every decision with enough context to reproduce it offline:

structure snapshot, regime features, costs, risk checks, and execution route. Treat every outage as a test of your reconciliation logic and your kill switch, not as an exception.

A.9.2 Operational Note 2

In live production, the difference between a good system and a brittle one is rarely the entry rule. It is consistency under stress: data outages, latency spikes, broker quirks, session transitions, and user misconfiguration. Implement defensive checks at every boundary: validate inputs, cap loops, sanitize payloads, and fail closed when uncertain. Store every decision with enough context to reproduce it offline: structure snapshot, regime features, costs, risk checks, and execution route. Treat every outage as a test of your reconciliation logic and your kill switch, not as an exception.

A.9.3 Operational Note 3

In live production, the difference between a good system and a brittle one is rarely the entry rule. It is consistency under stress: data outages, latency spikes, broker quirks, session transitions, and user misconfiguration. Implement defensive checks at every boundary: validate inputs, cap loops, sanitize payloads, and fail closed when uncertain. Store every decision with enough context to reproduce it offline: structure snapshot, regime features, costs, risk checks, and execution route. Treat every outage as a test of your reconciliation logic and your kill switch, not as an exception.

A.9.4 Operational Note 4

In live production, the difference between a good system and a brittle one is rarely the entry rule. It is consistency under stress: data outages, latency spikes, broker quirks, session transitions, and user misconfiguration. Implement defensive checks at every boundary: validate inputs, cap loops, sanitize payloads, and fail closed when uncertain. Store every decision with enough context to reproduce it offline: structure snapshot, regime features, costs, risk checks, and execution route. Treat every outage as a test of your reconciliation logic and your kill switch, not as an exception.

A.9.5 Operational Note 5

In live production, the difference between a good system and a brittle one is rarely the entry rule. It is consistency under stress: data outages, latency spikes, broker quirks, session transitions, and user misconfiguration. Implement defensive checks at every boundary: validate inputs, cap loops, sanitize payloads, and fail closed when uncertain. Store every decision with enough context to reproduce it offline: structure snapshot, regime features, costs, risk checks, and execution route. Treat every outage as a test of your reconciliation logic and your kill switch, not as an exception.

A.9.6 Operational Note 6

In live production, the difference between a good system and a brittle one is rarely the entry rule. It is consistency under stress: data outages, latency spikes, broker quirks, session transitions, and user misconfiguration. Implement defensive checks at every boundary: validate inputs, cap loops, sanitize payloads, and fail closed when uncertain. Store every decision with enough context to reproduce it offline: structure snapshot, regime features, costs, risk checks, and execution route. Treat every outage as a test of your reconciliation logic and your kill switch, not as an exception.

A.9.7 Operational Note 7

In live production, the difference between a good system and a brittle one is rarely the entry rule. It is consistency under stress: data outages, latency spikes, broker quirks, session transitions, and user misconfiguration. Implement defensive checks at every boundary: validate inputs, cap loops, sanitize payloads, and fail closed when uncertain. Store every decision with enough context to reproduce it offline: structure snapshot, regime features, costs, risk checks, and execution route. Treat every outage as a test of your reconciliation logic and your kill switch, not as an exception.

A.9.8 Operational Note 8

In live production, the difference between a good system and a brittle one is rarely the entry rule. It is consistency under stress: data outages, latency spikes, broker quirks, session transitions, and user misconfiguration. Implement defensive checks at every boundary: validate inputs, cap loops, sanitize payloads, and fail closed when uncertain. Store every decision with enough context to reproduce it offline: structure snapshot, regime features, costs, risk checks, and execution route. Treat every outage as a test of your reconciliation logic and your kill switch, not as an exception.

A.9.9 Operational Note 9

In live production, the difference between a good system and a brittle one is rarely the entry rule. It is consistency under stress: data outages, latency spikes, broker quirks, session transitions, and user misconfiguration. Implement defensive checks at every boundary: validate inputs, cap loops, sanitize payloads, and fail closed when uncertain. Store every decision with enough context to reproduce it offline: structure snapshot, regime features, costs, risk checks, and execution route. Treat every outage as a test of your reconciliation logic and your kill switch, not as an exception.

A.9.10 Operational Note 10

In live production, the difference between a good system and a brittle one is rarely the entry rule. It is consistency under stress: data outages, latency spikes, broker quirks, session transitions, and user misconfiguration. Implement defensive checks at every boundary: validate inputs, cap loops, sanitize payloads, and fail closed when uncertain. Store every decision with enough context to reproduce it offline: structure snapshot, regime features, costs, risk checks, and execution route. Treat every outage as a test of your reconciliation logic and your kill switch, not as an exception.

A.9.11 Operational Note 11

In live production, the difference between a good system and a brittle one is rarely the entry rule. It is consistency under stress: data outages, latency spikes, broker quirks, session transitions, and user misconfiguration. Implement defensive checks at every boundary: validate inputs, cap loops, sanitize payloads, and fail closed when uncertain. Store every decision with enough context to reproduce it offline: structure snapshot, regime features, costs, risk checks, and execution route. Treat every outage as a test of your reconciliation logic and your kill switch, not as an exception.

A.9.12 Operational Note 12

In live production, the difference between a good system and a brittle one is rarely the entry rule. It is consistency under stress: data outages, latency spikes, broker quirks, session transitions, and user

misconfiguration. Implement defensive checks at every boundary: validate inputs, cap loops, sanitize payloads, and fail closed when uncertain. Store every decision with enough context to reproduce it offline: structure snapshot, regime features, costs, risk checks, and execution route. Treat every outage as a test of your reconciliation logic and your kill switch, not as an exception.

A.9.13 Operational Note 13

In live production, the difference between a good system and a brittle one is rarely the entry rule. It is consistency under stress: data outages, latency spikes, broker quirks, session transitions, and user misconfiguration. Implement defensive checks at every boundary: validate inputs, cap loops, sanitize payloads, and fail closed when uncertain. Store every decision with enough context to reproduce it offline: structure snapshot, regime features, costs, risk checks, and execution route. Treat every outage as a test of your reconciliation logic and your kill switch, not as an exception.

A.9.14 Operational Note 14

In live production, the difference between a good system and a brittle one is rarely the entry rule. It is consistency under stress: data outages, latency spikes, broker quirks, session transitions, and user misconfiguration. Implement defensive checks at every boundary: validate inputs, cap loops, sanitize payloads, and fail closed when uncertain. Store every decision with enough context to reproduce it offline: structure snapshot, regime features, costs, risk checks, and execution route. Treat every outage as a test of your reconciliation logic and your kill switch, not as an exception.

A.9.15 Operational Note 15

In live production, the difference between a good system and a brittle one is rarely the entry rule. It is consistency under stress: data outages, latency spikes, broker quirks, session transitions, and user misconfiguration. Implement defensive checks at every boundary: validate inputs, cap loops, sanitize payloads, and fail closed when uncertain. Store every decision with enough context to reproduce it offline: structure snapshot, regime features, costs, risk checks, and execution route. Treat every outage as a test of your reconciliation logic and your kill switch, not as an exception.

A.9.16 Operational Note 16

In live production, the difference between a good system and a brittle one is rarely the entry rule. It is consistency under stress: data outages, latency spikes, broker quirks, session transitions, and user misconfiguration. Implement defensive checks at every boundary: validate inputs, cap loops, sanitize payloads, and fail closed when uncertain. Store every decision with enough context to reproduce it offline: structure snapshot, regime features, costs, risk checks, and execution route. Treat every outage as a test of your reconciliation logic and your kill switch, not as an exception.

A.9.17 Operational Note 17

In live production, the difference between a good system and a brittle one is rarely the entry rule. It is consistency under stress: data outages, latency spikes, broker quirks, session transitions, and user misconfiguration. Implement defensive checks at every boundary: validate inputs, cap loops, sanitize payloads, and fail closed when uncertain. Store every decision with enough context to reproduce it offline: structure snapshot, regime features, costs, risk checks, and execution route. Treat every outage as a test

of your reconciliation logic and your kill switch, not as an exception.

A.9.18 Operational Note 18

In live production, the difference between a good system and a brittle one is rarely the entry rule. It is consistency under stress: data outages, latency spikes, broker quirks, session transitions, and user misconfiguration. Implement defensive checks at every boundary: validate inputs, cap loops, sanitize payloads, and fail closed when uncertain. Store every decision with enough context to reproduce it offline: structure snapshot, regime features, costs, risk checks, and execution route. Treat every outage as a test of your reconciliation logic and your kill switch, not as an exception.

A.9.19 Operational Note 19

In live production, the difference between a good system and a brittle one is rarely the entry rule. It is consistency under stress: data outages, latency spikes, broker quirks, session transitions, and user misconfiguration. Implement defensive checks at every boundary: validate inputs, cap loops, sanitize payloads, and fail closed when uncertain. Store every decision with enough context to reproduce it offline: structure snapshot, regime features, costs, risk checks, and execution route. Treat every outage as a test of your reconciliation logic and your kill switch, not as an exception.

A.9.20 Operational Note 20

In live production, the difference between a good system and a brittle one is rarely the entry rule. It is consistency under stress: data outages, latency spikes, broker quirks, session transitions, and user misconfiguration. Implement defensive checks at every boundary: validate inputs, cap loops, sanitize payloads, and fail closed when uncertain. Store every decision with enough context to reproduce it offline: structure snapshot, regime features, costs, risk checks, and execution route. Treat every outage as a test of your reconciliation logic and your kill switch, not as an exception.

Appendix B: Additional Implementation Notes (Padding)

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.

Additional implementation notes: validate regime gating, enforce idempotency, track calibration drift, and test execution under simulated slippage and partial fills.